Python Mock Interview – 50 Questions with Answers

Theory • Scenario-Based • Coding • Student Management System

Part 1: Python Theory – 20 Questions

1. What is Python? What are its main features?

Python is a high-level, interpreted, general-purpose programming language. Features include simple syntax, dynamic typing, object-oriented programming, platform independence, a large standard library, open-source licensing, and support for web development, data science, AI/ML, and automation.

2. Difference between list, tuple, set, and dictionary

List: ordered, mutable, allows duplicates. Tuple: ordered, immutable, allows duplicates. Set: unordered collection of unique values. Dictionary: key-value pairs; keys are unique and dictionaries preserve insertion order in modern Python.

3. Mutable vs immutable data types

Mutable objects can be changed after creation, e.g. list, dictionary, and set. Immutable objects cannot be changed after creation, e.g. int, float, string, tuple, and bool.

4. Difference between == and is

== compares values. is compares object identity, meaning whether two references point to the same object.

5. Python built-in data types

Important types include int, float, complex, bool, str, list, tuple, set, frozenset, dict, NoneType, bytes, bytearray, and range.

6. What is type casting?

Type casting means converting one data type into another, such as int('100'), float(10), str(100), or list('Python').

7. append(), extend(), and insert()

append() adds one item at the end. extend() adds multiple elements from an iterable. insert(index, value) adds an item at a specified position.

8. remove(), pop(), and del

remove(value) removes a matching value. pop(index) removes and returns an item. del deletes an item, slice, or variable.

9. What is a dictionary?

A dictionary stores key-value pairs. Example: student={'name':'Salma','age':23}. Access with student['name']; add with student['marks']=85; update by assignment; delete with del.

10. What is list comprehension?

A concise way to create lists. Example: numbers = [i*2 for i in range(1,6)] produces [2,4,6,8,10].

11. Normal function vs lambda

A normal function is defined with def and can contain multiple statements. A lambda is an anonymous function intended for a small expression, e.g. add=lambda a,b:a+b.

12. Explain *args and **kwargs

*args collects multiple positional arguments into a tuple. **kwargs collects multiple keyword arguments into a dictionary.

13. Local, global, and nonlocal variables

Local variables exist inside a function. Global variables are defined outside functions. Nonlocal is used in nested functions to modify a variable from the enclosing function.

14. Exception handling

try contains risky code; except handles an exception; else runs when no exception occurs; finally runs regardless of whether an exception occurred.

15. Syntax errors vs runtime errors

A syntax error prevents Python from parsing the program, such as a missing colon. A runtime error occurs while the program is executing, such as ZeroDivisionError.

16. Module vs package

A module is generally a Python file containing code. A package is a directory used to organize related Python modules.

17. Purpose of if __name__ == '__main__'

It ensures a block runs when the file is executed directly, but not when the module is imported.

18. Iterators and generators

Iterators produce values one at a time using __iter__() and __next__(). Generators are commonly created with yield and produce values lazily, which is useful for large data.

19. break, continue, and pass

break exits a loop. continue skips the current iteration. pass does nothing and is used as a placeholder.

20. Shallow copy vs deep copy

A shallow copy creates a new outer object while nested objects may be shared. A deep copy recursively copies nested objects. copy.copy() and copy.deepcopy() are commonly used.

Part 2: Scenario-Based Questions – 15 Questions

21. Remove duplicate employee IDs while preserving order

Use a set to track values already seen and a list to preserve order.

22. User enters text instead of a number

Use try-except with int() and handle ValueError so the program does not crash.

23. Employees with salary greater than ■50,000

Loop through dictionary items and print entries where salary > 50000.

24. Large list is slow

Profile first, then consider better algorithms/data structures, sets or dictionaries for lookups, generators, batching, and avoiding unnecessary nested loops.

25. CSV with millions of records

Do not normally load the whole file into memory. Process rows incrementally or use chunking for data-analysis workloads.

26. API returns None

Use safe access such as data.get('name') and explicitly handle None or provide an appropriate default.

27. Count word frequency

Use a dictionary mapping each word to its count; collections.Counter is also designed for this task.

28. File does not exist

Catch FileNotFoundError and show a meaningful message or recovery option.

29. Same function name in different files

Import the modules and call module.function(), or use import aliases such as from file1 import display as display1.

30. Find even numbers in one million numbers

Use a list comprehension if all results are needed in memory; use a generator if results can be processed lazily.

31. Invalid email address

Validate at the application/input boundary before processing or storing the data. Production validation should be more robust than checking only for '@'.

32. Filter salary > ■40,000

If records are stored in a database, filter in SQL using WHERE salary > 40000 to reduce transferred data and application memory usage.

33. Temporarily store login information

For a simple in-memory example, a dictionary can hold non-sensitive state. Real web applications should use sessions or another secure mechanism. Never store passwords as plain text.

34. Repeated code in five places

Follow DRY (Don't Repeat Yourself): extract repeated logic into reusable functions or modules.

35. Production exceptions suddenly appear

Check logs and traceback, identify when it started, inspect recent changes, reproduce if possible, check dependencies and data, fix the root cause, test, deploy, and monitor.

Part 3: Python Coding Questions – 15 Questions

36. Reverse a String

```
text = "Python"
reverse = ""
for char in text:
    reverse = char + reverse
print(reverse)
# Output: nohtyP
```

37. Palindrome

```
text = "madam"
reverse = ""
for char in text:
    reverse = char + reverse
if text == reverse:
    print("Palindrome")
else:
    print("Not Palindrome")
```

38. Find Even and Odd Numbers

```
numbers = [10, 15, 20, 25, 30, 35]
even = []
odd = []
for num in numbers:
    if num % 2 == 0:
        even.append(num)
    else:
        odd.append(num)
print("Even:", even)
print("Odd:", odd)
```

39. Largest Number Without max()

```
numbers = [25, 10, 45, 30, 15]
largest = numbers[0]
for num in numbers:
    if num > largest:
        largest = num
print("Largest:", largest)
```

40. Remove Duplicates Without set()

```
numbers = [10, 20, 10, 30, 20, 40]
result = []
for num in numbers:
    if num not in result:
        result.append(num)
print(result)
```

41. Character Frequency

```
text = "programming"
frequency = {}
for char in text:
    frequency[char] = frequency.get(char, 0) + 1
for char, count in frequency.items():
    print(char, "->", count)
```

42. Second Largest Number

```
numbers = [10, 50, 20, 80, 60]
largest = float("-inf")
second = float("-inf")
for num in numbers:
    if num > largest:
        second = largest
        largest = num
    elif largest > num > second:
        second = num
print("Second largest:", second)
# Output: 60
```

43. Prime Number

```
def check_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False
    return True

num = 17
print("Prime" if check_prime(num) else "Not Prime")
```

44. Fibonacci Series

```
n = 7
a, b = 0, 1
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
# Output: 0 1 1 2 3 5 8
```

45. Factorial Using Recursion

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
# Output: 120
```

46. Word Frequency

```
text = "python is easy and python is powerful"
words = text.split()
frequency = {}
for word in words:
    frequency[word] = frequency.get(word, 0) + 1
for word, count in frequency.items():
    print(word, "->", count)
```

47. Find Missing Number

```
numbers = [1, 2, 3, 5, 6]
n = 6
expected_sum = n * (n + 1) // 2
```

```
actual_sum = 0
for num in numbers:
    actual_sum += num
print("Missing number:", expected_sum - actual_sum)
# Output: 4
```

48. Sort Without sort() or sorted()

```
numbers = [5, 2, 8, 1, 3]
n = len(numbers)
for i in range(n):
    for j in range(0, n-i-1):
        if numbers[j] > numbers[j+1]:
            numbers[j], numbers[j+1] = numbers[j+1], numbers[j]
print(numbers)
# Output: [1, 2, 3, 5, 8]
```

49. Employee Salary Problem

```
employees = {
    "Rahul": 45000,
    "Priya": 60000,
    "Arun": 35000,
    "Sneha": 75000
}
for name, salary in employees.items():
    if salary > 50000:
        print(name, salary)
# Output:
# Priya 60000
# Sneha 75000
```

50. Student Management System

```
students = {}

def add_student():
    student_id = input("Enter Student ID: ")
    if student_id in students:
        print("Student ID already exists.")
        return
    name = input("Enter Name: ")
    age = int(input("Enter Age: "))
    course = input("Enter Course: ")
    marks = float(input("Enter Marks: "))
    students[student_id] = {
        "name": name, "age": age,
        "course": course, "marks": marks
    }
    print("Student added successfully.")

def view_students():
    if not students:
        print("No students found.")
        return
    for student_id, student in students.items():
        print("\nStudent ID:", student_id)
        print("Name:", student["name"])
        print("Age:", student["age"])
        print("Course:", student["course"])
        print("Marks:", student["marks"])

def search_student():
    student_id = input("Enter Student ID: ")
    if student_id in students:
        student = students[student_id]
        print("\nStudent Found")
        print("Name:", student["name"])
        print("Age:", student["age"])
        print("Course:", student["course"])
        print("Marks:", student["marks"])
    else:
        print("Student not found.")

def update_student():
```

```
    student_id = input("Enter Student ID: ")
    if student_id not in students:
        print("Student not found.")
        return
    student = students[student_id]
    student["name"] = input("Enter new name: ")
    student["age"] = int(input("Enter new age: "))
    student["course"] = input("Enter new course: ")
    student["marks"] = float(input("Enter new marks: "))
    print("Student updated successfully.")

def delete_student():
    student_id = input("Enter Student ID: ")
    if student_id in students:
        del students[student_id]
        print("Student deleted successfully.")
    else:
        print("Student not found.")

while True:
    print("\n===== STUDENT MANAGEMENT SYSTEM =====")
    print("1. Add Student")
    print("2. View Students")
    print("3. Search Student")
    print("4. Update Student")
    print("5. Delete Student")
    print("6. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_student()
    elif choice == "2":
        view_students()
    elif choice == "3":
        search_student()
    elif choice == "4":
        update_student()
    elif choice == "5":
        delete_student()
    elif choice == "6":
        print("Thank you!")
        break
    else:
        print("Invalid choice.")
```

Interview Tip

Do not memorize only the code. Be ready to explain why you selected each data structure, function, loop, exception-handling technique, or database operation.